

Disparos do Mestre

Revisão técnica: bugs, melhorias visuais e novas funcionalidades

Resumo

Revisei o código completo do projeto (backend Node/Express/SQLite e frontend vanilla JS) na pasta local. O sistema está bem estruturado para um MVP: schema relacional coerente, scheduler com restauração de jobs após restart, e UI dark mode consistente. Encontrei, no entanto, alguns bugs reais que vão pegar você no primeiro uso fora do "localhost", além de features já parcialmente construídas no backend (grupos de canais, canal ativo/inativo) que nunca chegaram na interface.

O documento está dividido em três partes: bugs e problemas técnicos (por severidade), melhorias visuais/UX, e sugestões de novas funcionalidades.

1. Bugs e problemas técnicos

1.1 Críticos — vão quebrar o sistema em produção

Severidade	Onde	Problema	S
Crítico	frontend/assets/app.js:3 e campaigns.html:307	A URL da API está hardcoded como http://localhost:3131. Se você acessar o painel de outro dispositivo, ou fizer o deploy via Docker em um servidor/VPS, o frontend continua tentando chamar localhost do navegador do cliente — nada funciona.	T A o C ir 3 fi 4 "
Crítico	backend/routes/channels.js (rota /:id/verify) + frontend (innerHTML)	XSS armazenado real: ao clicar em "Testar Conexão", o backend copia o título do chat do Telegram (chat.title) direto para o campo "name" do canal, sem sanitizar. Esse nome depois é injetado via innerHTML em channels.html e campaigns.html sem escape. Qualquer pessoa com acesso para renomear o grupo/canal no Telegram (não precisa ser você) pode colocar um título tipo e ele executa no seu painel.	E a (H c n n h s

1.2 Altos — perda de dados, falhas silenciosas

Severidade	Onde	Problema	S
Alto	backend/scheduler.js — cancelJob() e rota DELETE /api/campaigns/:id	Ao excluir uma campanha, o registro é apagado do banco (e o CASCADE limpa scheduled_jobs), mas o cron job ativo em memória (activeCronJobs) nunca é parado. O node-cron continua rodando indefinidamente tentando disparar uma campanha que não existe mais — vazamento de memória/CPU que só some ao reiniciar o servidor.	A s c c c

Severidade	Onde	Problema	S
Alto	backend/routes/campaigns.js (POST/PUT/DELETE) + backend/data/uploads	Imagens enviadas nunca são removidas do disco: ao trocar a imagem de uma campanha (PUT) o arquivo antigo fica órfão, e ao excluir a campanha (DELETE) a imagem também não é apagada. Com o tempo a pasta uploads só cresce.	A a c
Alto	backend/scheduler.js — scheduleJob()	Se o cron_expr for inválido, o código só faz console.error e retorna — o job fica com status "pending" para sempre no banco, e o usuário nunca é avisado que o agendamento não vai funcionar.	V e c d i
Alto	backend/bot.js — sendCampaignToChannel()	Quando a campanha tem imagem, o texto vira "caption" da foto — e legendas do Telegram têm limite de 1024 caracteres (contra 4096 de uma mensagem de texto puro). Se o texto passar disso, o Telegram rejeita o envio para TODOS os canais e a campanha falha inteira, sem aviso prévio no editor.	A e d 4

1.3 Médios

Severidade	Onde	Problema	S
Médio	backend/scheduler.js — dispatchJob()	Para jobs recorrentes, o campo next_run_at é sempre gravado como "agora + 1 minuto", independente da expressão cron real. O dashboard mostra um "próximo disparo" que quase sempre está errado.	C p li n
Médio	backend/bot.js — sendCampaignToChannel() (loop em scheduler.js)	Os envios para múltiplos canais acontecem em sequência sem nenhum delay. A API do Telegram tem limite de ~1 msg/segundo por chat e ~30/segundo no total — campanhas para muitos canais podem tomar erro 429 (rate limit) no meio do disparo.	A e 4 re
Médio	frontend/assets/style.css — .tg-preview	A regra usa border-radius: var(--radius-md), mas essa variável não existe no :root (só existem --radius-sm, --radius, --radius-lg, --radius-xl). O valor cai para o default e o preview do Telegram perde os cantos arredondados.	T ra
Médio	frontend/style.css (.sidebar.open) — nenhum JS correspondente	O CSS já prevê um menu lateral retrátil no mobile (classe .sidebar.open), mas não existe nenhum botão de hambúrguer nem JS que adicione essa classe. Em telas pequenas a sidebar simplesmente some e não há como abri-la — navegação quebrada no celular.	A to C n
Médio	Toda a aplicação — rotas da API	Não existe autenticação nenhuma. Qualquer pessoa que acesse a porta 3131 (inclusive na rede local, ou se o Docker for exposto sem proxy) pode ver o token do bot mascarado, cadastrar canais, disparar campanhas e apagar mensagens.	P (o p D

1.4 Menores / polimento

Severidade	Onde	Problema	S
Baixo	frontend/campaigns.html —	Comentário no próprio código admite: "NOTE: Handle removing	E

Severidade	Onde	Problema	S
	saveCampaign()	existing image if needed, for simplicity omitting here" — hoje não dá para remover a imagem de uma campanha já salva, só trocar por outra.	ti C a
Baixo	frontend/campaigns.html — addBtnRow() / btn-builder	A interface só permite um botão por linha, mas o schema/backend já suporta múltiplos botões lado a lado na mesma linha (array de arrays). Fica sem usar.	P n a a
Baixo	frontend/campaigns.html — setSchedTab()	A função usa a variável global implícita "event" (event.target) em vez de receber o evento como parâmetro. Funciona no Chrome mas é uma prática frágil/depreciada que pode falhar em outros contextos.	P o e
Baixo	backend/routes/channels.js e campaigns.js	channel_groups tem API completa (criar, listar, apagar grupo) mas nenhuma tela usa isso — nem para agrupar canais, nem para filtrar na hora de montar uma campanha. Feature morta.	C d f

2. Melhorias visuais e de UX

- O visual dark mode já está bem cuidado (gradientes sutis, glow nos cards, tipografia Inter) — a base é boa. As sugestões abaixo são incrementais, não uma reforma.
- Menu mobile quebrado (ver bug acima) é a prioridade, senão o painel não funciona em celular/tablet.
- Adicionar contador de caracteres visível no editor de mensagem (atual/limite), já mencionado como bug — também é uma melhoria de UX importante.
- Os estados vazios (empty-state) já têm CSS pronto para ícone + título + descrição, mas as telas só usam texto simples ("Nenhum canal encontrado."). Vale usar o componente completo para dar mais contexto, com um botão de ação direto (ex: "+ Adicionar Canal" dentro do empty state).
- O seletor de canais dentro do editor de campanha não tem busca (channels.html tem, campaigns.html não) — incômodo se você tiver muitos canais cadastrados.
- Falta indicador de "salvando..." / desabilitar botão durante o clique em Salvar Campanha e Disparar — hoje dá para clicar várias vezes rápido e dependendo da rede isso poderia disparar duas vezes.
- O modal de confirmação de exclusão usa o confirm() nativo do navegador, que quebra o visual dark mode cuidadosamente construído. Um modal de confirmação customizado (reaproveitando o CSS de modal que já existe) ficaria mais consistente.
- Toggle visual para ativar/desativar canal direto na listagem (hoje o campo "active" existe no banco e é usado pelo scheduler, mas não tem controle na tela de Canais).
- A imagem de fundo do preview do Telegram é carregada de um domínio externo (web.telegram.org) — se cair ou for bloqueado, o preview perde o fundo. Melhor hospedar um padrão local ou usar CSS puro (gradiente) para não depender de terceiros.
- A fonte Inter vem via Google Fonts CDN — se o ambiente Docker não tiver saída de internet, a fonte não carrega. Vale considerar hospedar o .woff2 localmente para deploys isolados.

3. Sugestões de novas funcionalidades

3.1 Confiabilidade de envio

- Reenvio automático (retry com backoff) para canais que falharem, com número de tentativas configurável.

- Fila com controle de taxa (rate limiting) respeitando os limites do Telegram, para campanhas com muitos canais não tomarem 429.
- Envio de teste: botão "Enviar teste para mim" que manda a campanha só para um chat/canal de teste antes do disparo real para todos.
- Notificação (e-mail, ou mensagem do próprio bot para você no Telegram) quando um job agendado falhar.

3.2 Gestão de canais e campanhas

- Usar de verdade o recurso de grupos de canais já existente no backend: permitir selecionar "todos os canais do grupo X" na hora de montar uma campanha, em vez de marcar um por um.
- Biblioteca de mídia: reaproveitar imagens já enviadas em campanhas anteriores, em vez de fazer upload de novo toda vez.
- Duplicar campanha existente (clonar texto, imagem e botões como ponto de partida para uma nova).
- Suporte a álbuns (múltiplas imagens) e vídeos/documentos, não só uma imagem única — o Telegram Bot API já suporta sendMediaGroup e sendVideo.
- Templates de mensagem reutilizáveis (ex: "Promoção", "Aviso", "Novidade") para não recomeçar do zero.

3.3 Métricas e acompanhamento

- Rastreamento de cliques nos botões inline: usar uma URL de redirecionamento própria (ex: seusite.com/r/{id}) que registra o clique e depois redireciona para o link real — hoje não há como saber se alguém clicou.
- Taxa de sucesso por canal ao longo do tempo, para identificar canais problemáticos (bot removido, sem permissão, etc.) antes de tentar disparar de novo.
- Exportar histórico de envios para CSV/Excel.

3.4 Acesso e segurança

- Login simples (usuário/senha ou ao menos Basic Auth) antes de expor o painel fora do localhost — hoje qualquer um na rede tem controle total.
- Perfis de acesso (ex: um operador que só monta campanhas, sem poder trocar o token do bot ou apagar canais).

3.5 Fuso horário

- Hoje o agendamento "Uma vez" usa a hora local do navegador convertida para ISO, e o cron recorrente roda no fuso horário do servidor — se você mexer no painel de um fuso diferente de onde o servidor Docker está rodando, os horários não batem. Vale deixar explícito qual fuso está sendo usado, ou permitir escolher.

Revisão feita lendo o código-fonte completo do projeto na pasta conectada. Nenhuma alteração foi feita nos arquivos — este documento é só o diagnóstico.